SOFTWARE ENGINEERING

Assignment 1

SIMATS ENGINEERING

CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 1

CO1: Apply agile methodologies and project management techniques such as scale and lean to effectively manage software projects using industry-standard tools like Jira.

Assignment Title: Design, Simulate and Validate an Agile Delivery Strategy for the Omnichannel Order Fulfilment Platform

Nature of Assignment: Individual **Duration:** One week **Total Marks:** 50

Submission Date: _____

Scenario – Agile Delivery Planning for the Order Fulfilment Platform

A retailer sells through a website, a mobile app and twelve physical stores. It plans an integrated order fulfilment platform to reduce failed orders, slow refunds and unclear order ownership. Warehouse pickers prioritize accurate stock reservation and pick lists, support agents prioritize order visibility and fast return handling, and finance reviewers require auditable refunds and payment reconciliation. A seven-member team must select a feasible eight-week release before a fixed festival-sale date despite uncertain capacity, limited stakeholder time and delayed payment-gateway access. Create and validate an Agile delivery strategy connecting these priorities to a justified backlog, team responsibilities, Lean flow controls and sprint goals. Revise the strategy when regions, staffing and coordination needs change, while protecting a complete order-to-delivery-to-refund journey.

Planning conditions for the representative release are stated below.

Observation	Scenario planning condition
Operational scope	Checkout, stock reservation, payment, picking, dispatch, tracking, returns, refunds and support lookup
Users	Customer, warehouse picker, support agent, store manager, finance reviewer and administrator
Reference data	Synthetic products, synthetic orders and simulated stock movements; no real customer or card data
Integration constraint	Payment gateway sandbox may be unavailable; preserve local order records and retry payment requests safely without double charging

Validation need	Verify stock, order status, price, authorization, persistence and refund audit behaviour

The retailer proposes an omnichannel order fulfilment platform. Its intended modules are a storefront catalogue, cart and checkout, payment integration, inventory synchronization, order tracking, a warehouse picking dashboard, returns and refunds, a support console and sales analytics. The system should connect a customer order to stock reservation, payment confirmation, picking, dispatch, tracking, delivery and, where needed, an authorized return and refund.

A seven-member cross-functional team plans an initial release within eight weeks using four two-week sprints. There is no established velocity. Test access to the payment gateway sandbox begins in week four, and a barcode-scanner test device arrives in week five. Warehouse and finance stakeholders can attend clarification and review meetings for ninety minutes per week. These are assignment planning assumptions.

Principal users are customers, warehouse pickers, support agents, store managers, finance reviewers and administrators. They need different permissions. Pickers require accurate pick lists; support agents require readable order and return status; finance reviewers require auditable refund approvals; engineers require traceable links between orders, payments and releases. Automation must preserve human responsibility for refund approval and stock corrections.

Define workload, service-level policies and acceptance thresholds as explicit planning assumptions. Dashboard values, stock counts and screenshots are illustrative evidence of displayed behaviour, not proof of production accuracy, performance, live inventory control or secure payment handling. Verify claims through repeatable checks. Distinguish deterministic rules, such as fixed low-stock thresholds, from trained machine-learning forecasting, and label simulations and proposed extensions accurately.

Design Challenge:

Management requests all nine modules in the initial release. Pickers prioritize stock reservation, pick lists and dispatch; support agents prioritize order tracking and return handling; finance reviewers prioritize refund approval and payment reconciliation. Gateway access and limited review time constrain delivery. A broad release may leave every workflow incomplete, while a storefront-only release may fail to reduce failed or delayed orders.

Design, simulate and validate an Agile project delivery strategy using a project-management tool of your choice that produces a coherent and achievable first release under these conditions.

Develop alternative delivery approaches and create a justified strategy connecting backlog priorities to the identified operational problems. Establish team responsibilities, capacity assumptions, dependency sequencing, stakeholder review arrangements and Lean practices for controlling waiting, rework and work in progress.

Investigate at least two suitable project-management tools, such as Jira, Trello, Azure DevOps or GitHub Projects. Select one and justify its suitability for your strategy, considering planning, workflow management, collaboration, reporting, accessibility and cost. Explain any limitations and how you will address them.

Before simulation, predict the principal delivery bottleneck and define measurable criteria for judging the strategy. Use the resulting evidence to revise your plan and defend the features retained, deferred or resequenced. Your final design must explain how the team will deliver, why the selected release should improve fulfilment, and how the chosen tool supports the strategy.

TASK 1 – Investigate the Problem and Define Delivery Outcomes 5 Marks

Investigate the platform from the perspectives of customers, warehouse pickers, support agents, finance reviewers and the development team.

- Formulate at least three questions to investigate about priorities, workflow problems or delivery constraints.
- Obtain initial evidence through a small stakeholder discussion, questionnaire or documented review of the existing process. If access is unavailable, use clearly identified simulated stakeholder responses.
- Define the product goal, first-release scope and exclusions.
- Establish at least three measurable delivery-success criteria.
- Examine technical, operational and financial feasibility at a level appropriate to project planning.

Evidence Expected

Investigation questions, stakeholder findings or labelled simulated responses, a problem statement, scope boundaries, feasibility observations and measurable success criteria.

TASK 2 – Design and Justify the Project Management Approach 6 Marks

Develop two meaningfully different delivery approaches for the scenario. Consider Scrum, Lean flow management, XP practices or a justified combination.

- For each approach, explain how work would be selected, coordinated and reviewed.
- Identify its suitability for changing requirements and cross-functional work.
- Examine likely limitations, overheads and delivery risks.
- Select an approach and create a tailored operating model defining responsibilities, planning activities, review points, escalation procedures and stakeholder participation.
- Explain how Lean principles will help reduce waiting, unnecessary handovers, rework or excessive work in progress.

Evidence Expected

Comparison of two approaches, a justified selection and an original operating model with explicit Lean practices.

TASK 3 – Construct a Prioritized Product Backlog in the Selected Tool 8 Marks Create a project in your selected management tool and develop a backlog containing:

- At least four epics and twelve user stories covering order management, inventory and picking, payments and refunds, and tracking or support. Justify which modules are retained or deferred.
- User stories with clear acceptance criteria.
- Priorities, estimates, dependencies and responsible roles.
- A justified prioritization method.
- A distinction between essential first-release features and deferred features.
- A Definition of Ready and a Definition of Done appropriate to the project. Incorporate relevant privacy, accessibility and quality expectations into the backlog or completion criteria.
- An explanation of at least two decisions where competing stakeholder requests influenced your priorities.

Evidence Expected
Selected-tool backlog screenshots or export, story acceptance criteria, prioritization rationale, dependencies, and Definitions of Ready and Done.

TASK 4 – Create a Capacity-Based Release Plan and Delivery Workflow 7 Marks Design the four-sprint release plan using explicit assumptions about team availability and capacity.

- Define a goal for each sprint.
- Select work that fits the assumed capacity.
- Explain your estimation method and avoid treating story points as fixed hours.
- Represent dependencies and cross-functional responsibilities, including the week-four gateway and week-five scanner constraints.
- Create a development workflow in the selected tool with statuses, transition rules and completion checks. Distinguish this workflow from the operational order lifecycle.
- Set and justify work-in-progress limits where applicable.
- Define how blocked work, urgent requests and unfinished stories will be handled.
- Show how the selected features collectively achieve the product goal.

Evidence Expected
Release plan, capacity assumptions, sprint goals, selected-tool board or workflow, work-in-progress policies and dependency-management decisions.

TASK 5 – Simulate Delivery and Validate Your Design 7 Marks
Simulate the first sprint using a documented sequence of at least ten work events, such as starting a story, identifying a dependency, encountering a defect, receiving clarification or completing a review.

- Before simulation, predict where a bottleneck is most likely to occur.
- Identify which planning or workflow rule should help control it and what evidence would indicate that the rule is effective.
- Update the selected management tool to reflect the simulated events.
- Record suitable indicators such as completed versus planned work, blocked time, cycle time, work in progress or reopened stories.
- Use the results to identify a limitation in your original plan and create an improvement for the next sprint.

Evidence Expected
Prediction, event log, management-tool updates, at least three delivery indicators, interpretation of results and a revised planning or workflow decision.

All simulated results must be clearly labelled. Do not present simulated activity as actual team delivery.

TASK 6 – Redesign Delivery Under Changed Conditions 12 Marks Changed Requirement
At the end of the first simulated sprint, the organization introduces the following changes:

- Two additional regional warehouses must use the platform.
- A second seven-member development team joins the project.
- The payments engineer from the original team becomes unavailable for the remaining period.
- Region-specific tax, shipping and refund-approval rules are now required.

- The festival-sale release deadline remains unchanged.

Create a revised strategy that coordinates both teams, preserves a useful first release and manages the new constraints.

Component	Required Work	Marks
A. Diagnose the impact	Identify affected assumptions, capacity, scope, dependencies and risks using evidence from your original plan.	2
B. Define redesign objectives	Establish measurable objectives for the revised strategy, including achievable scope and coordination expectations.	2
C. Create the revised strategy	Redesign backlog priorities, team responsibilities, release sequencing and dependency coordination. Incorporate justified scaled Agile and Lean practices.	4
D. Validate the redesign	Simulate at least five additional events and compare the revised strategy with the original using equivalent indicators.	2
E. Defend the decision	Explain benefits, limitations and trade-offs. Show why the revised strategy remains feasible under the unchanged deadline.	2

You may use selected SAFe-inspired coordination practices or another justified scaled Agile approach. Explain what you adopt and why it fits this project.

Evidence Expected

Impact analysis, measurable redesign objectives, revised management-tool artefacts, cross-team coordination design, comparative simulation evidence and an evidence-based defence.

TASK 7 – Individual Technical Reflection 5 Marks Explain:

- Which project-management decision you consider most significant and why.
- How your evidence influenced a change to the original strategy.
- Where your final design remains vulnerable.
- How you addressed an ethical concern involving privacy, workload, accessibility or transparency.
- What you would investigate next before using the strategy with a real development team.

Evidence Expected

A concise individual reflection demonstrating understanding of the created strategy and its limitations. Refer to your own artefacts and findings.

MARK DISTRIBUTION

Task	Assessed Area	Marks

Task 1	Problem investigation and measurable delivery outcomes	5
Task 2	Original Agile and Lean operating model	6
Task 3	Prioritized Jira backlog and acceptance criteria	8
Task 4	Capacity-based release plan and workflow	7
Task 5	Delivery simulation and evidence-based improvement	7
Task 6	Day-4 Design Challenge – scaled delivery redesign and validation	12
Task 7	Individual technical reflection	5
TOTAL		50

Submission Requirements

- Assignment report covering investigation findings, scope, delivery approaches and project-management tool selection.
- Project export or screenshots showing epics, user stories, acceptance criteria, priorities, estimates and dependencies.
- Definition of Ready, Definition of Done, release plan, capacity assumptions and selected-tool workflow with work-in-progress limits.
- Sprint 1 simulation: prediction, event log, management-tool updates and delivery indicators, all clearly labelled as simulated.
- Day-4 redesign artefacts: impact analysis, redesign objectives, revised backlog, cross-team coordination design and comparative simulation evidence.
- Individual reflection and acknowledgement of sources and tools used.

Assessment Verification

Students may be asked to explain a prioritization decision, trace a story through the selected workflow, justify capacity or a work-in-progress limit, interpret an indicator, or revise the plan when a constraint changes.

Overall Assessment Expectation

Investigate → Design → Plan → Simulate → Validate → Redesign → Defend

The final submission must demonstrate an original, evidence-supported delivery strategy aligned with CO1, BTL 5 – Evaluate and BTL 6 – Create.

Software Engineering
Assignment 1

Agile Delivery Strategy for the Omnichannel Order Fulfilment Platform

Student Name: L .ASIF ALI KHAN

Register Number: 192521009

Programme and Section: B.TECH(IT)

Submission Date: _____

CO1: Apply agile methodologies and project management techniques such as scale and lean to effectively manage software projects using industry-standard tools like Jira.

This report develops a Scrum delivery strategy with Lean flow controls and Jira planning. It prioritizes a complete order-to-refund journey, defines capacity and dependencies, evaluates a first-sprint simulation, and redesigns delivery for two teams and additional regional warehouses.

Method: Stakeholder responses, sprint events and redesign indicators in this report are simulated and labelled as such. The figures are illustrative Jira-style mock-ups; replace them with screenshots from your own project. Story points and sprint indicators are planning figures, not measured team performance.

Problem Statement

A retailer plans an integrated order fulfilment platform to reduce failed orders, slow refunds and unclear order ownership. Pickers prioritize accurate stock reservation, support agents prioritize order visibility and returns, and finance reviewers require auditable refunds. A seven-member team must select a feasible eight-week release despite uncertain capacity, limited stakeholder time and delayed gateway access. Create and validate an Agile delivery strategy connecting these priorities to a justified backlog, team responsibilities, Lean flow controls and sprint goals. Revise the strategy when regions, staffing and coordination needs change, while protecting a complete order-to-refund journey.

TASK 1 – Investigate the Problem and Define Delivery Outcomes

Simulated Stakeholder Findings

The following findings are simulated for assignment planning. They are not actual stakeholder interview results.

Stakeholder	Identified Problem	Expected Solution
Customer	Order status is unclear	Real-time order tracking
Warehouse Picker	Incorrect stock information	Accurate stock reservation and pick lists

Support Agent	Delayed return handling	Support dashboard
Finance Reviewer	Refund approval is difficult to audit	Refund approval and audit records
Store Manager	Stock differs between stores	Inventory synchronization
Development Team	Payment gateway access is delayed	Sandbox simulation and safe retry mechanism

Problem Statement

The retailer requires an integrated order fulfilment platform to improve order accuracy, inventory management, payment processing, warehouse operations, order tracking, and refund handling.

The development team must deliver a feasible first release within eight weeks using Agile methodologies. The platform must provide a complete order-to-delivery-to-refund journey while managing uncertain capacity, delayed payment gateway access, and limited stakeholder availability.

Product Goal

To develop a reliable omnichannel order fulfilment platform that reduces failed orders, improves inventory accuracy, provides transparent order tracking, and enables secure and auditable refund processing.

First-Release Scope

Included features:

- Customer login and authentication.
- Product catalogue and cart.
- Order placement and stock reservation.
- Payment integration with safe retry.
- Warehouse picking and dispatch.
- Order tracking.
- Return requests.
- Authorized refund processing.
- Support order lookup.

Excluded features:

- Advanced AI-based demand forecasting.
- Complex sales analytics.
- Loyalty and reward programs.
- Fully automated refund approval.
- Advanced warehouse robotics.

Measurable Success Criteria

Criteria	Target
Order completion	At least 95% of valid test orders complete the defined workflow
Stock reservation	No overselling in 100 simulated concurrent-order checks
Payment safety	Zero duplicate charges in retry tests
Refund audit	100% of simulated refunds contain approval and audit records
Order tracking	Status updates are visible within 5 seconds in the test environment

Feasibility Analysis

Technical feasibility: The platform can be developed using Java, MySQL, REST APIs, and a web-based frontend. Payment processing can initially be simulated.

Operational feasibility: Warehouse pickers, support agents, and finance reviewers can use role-based dashboards.

Financial feasibility: Open-source technologies and Jira's available free offering can reduce initial project costs.

Time feasibility: Four two-week sprints are suitable for developing a minimum viable release within eight weeks.

TASK 2 – Design and Justify the Project Management Approach

Comparison of Agile Approaches

Two Agile approaches are considered for the project.

Feature	Scrum with Lean	Kanban
Planning	Fixed two-week sprints	Continuous planning
Work selection	Sprint backlog	Pull-based workflow
Review	Sprint review	Continuous review
Change handling	Changes normally enter the next sprint	Changes can enter when capacity is available
Work tracking	Sprint board and burndown	Kanban board and cycle time
Best suited for	Structured release planning	Continuous delivery
Limitation	Sprint changes may be difficult	Release goals may become less predictable

Selected Approach

Selected Methodology: Scrum with Lean Flow Management

Scrum is selected because the project has a fixed eight-week release deadline and requires four two-week sprints.

Lean practices are integrated to reduce waiting time, unnecessary handovers, defects, and excessive work in progress.

Scrum Team Responsibilities

The seven-member team consists of:

Role	Responsibility
Product Owner	Defines priorities and product goals
Scrum Master	Facilitates Scrum and removes impediments
Backend Developer 1	Order management and inventory
Backend Developer 2	Payment and refund processing
Frontend Developer 1	Customer storefront
Frontend Developer 2	Warehouse and support dashboards
QA Engineer	Testing, validation, and defect tracking

Team members collaborate across responsibilities when necessary.

Operating Model

The project follows a two-week sprint cycle.

Activity	Frequency
Sprint Planning	First day of each sprint
Daily Scrum	15 minutes daily
Backlog Refinement	Weekly
Sprint Review	Last day of sprint
Sprint Retrospective	Last day of sprint
Stakeholder Clarification	90 minutes per week

Escalation Procedure

1. A team member identifies a blocker.
2. The blocker is recorded in Jira.
3. The Scrum Master attempts to resolve it.

4. Technical dependencies are escalated to the technical lead.
5. Scope conflicts are escalated to the Product Owner.
6. Critical unresolved issues are discussed with stakeholders.

Lean Practices

- Limit Work in Progress: Prevent too many stories from being developed simultaneously.
- Reduce Waiting: Identify blocked stories quickly.
- Reduce Rework: Use clear acceptance criteria.
- Eliminate Waste: Avoid unnecessary documentation and duplicate activities.
- Continuous Improvement: Use sprint retrospectives to improve the process.

Project Management Tool Comparison

Feature	Jira	Trello
Backlog management	Advanced	Basic
Sprint planning	Supported	Limited
Workflow customization	Advanced	Simple
Reporting	Burndown, velocity, reports	Basic
Collaboration	Strong	Strong
Suitability	High	Moderate

Conclusion: Jira is selected because it supports epics, user stories, sprint planning, dependencies, workflow customization, and reporting.

TASK 3 – Construct a Prioritized Product Backlog in Jira

Product Backlog

The backlog contains four epics and twelve user stories.

Epic 1: Order and Inventory Management

ID	User Story	Priority	Estimate	Owner
US01	As a customer, I want to log in securely so that I can access my account.	High	3	Backend Developer
US02	As a customer, I want to place an order so that I can purchase products.	Highest	5	Backend Developer
US03	As a system, I want to reserve stock so that overselling is prevented.	Highest	5	Backend Developer

Acceptance Criteria:

- Valid users can log in.
- Customers can place orders with valid products.
- Stock is reserved before order confirmation.
- Insufficient stock prevents order confirmation.

Epic 2: Payment and Refund Governance

ID	User Story	Priority	Estimate	Owner
US04	As a customer, I want to make payments securely.	Highest	5	Payment Developer
US05	As a system, I want to retry failed payments safely.	Highest	5	Payment Developer
US06	As a finance reviewer, I want to approve refunds with an audit trail.	High	5	Finance/Backend

Acceptance Criteria:

- Payment requests use unique transaction identifiers.
- Retrying a payment does not create duplicate charges.
- Refunds require authorized approval.
- Refund records contain amount, approver, time, and order reference.

Epic 3: Warehouse Fulfilment and Tracking

ID	User Story	Priority	Estimate	Owner
US07	As a picker, I want an accurate pick list.	Highest	5	Warehouse Developer
US08	As a warehouse manager, I want to dispatch packed orders.	High	3	Warehouse Developer
US09	As a customer, I want to track my order.	High	3	Frontend Developer

Acceptance Criteria:

- Pick lists show reserved products and quantities.
- Dispatch is permitted only after picking and packing.
- Customers can view order status.
- Status changes are stored in the database.

Epic 4: Returns and Support

ID	User Story	Priority	Estimate	Owner
US10	As a customer, I want to request a return.	High	3	Frontend Developer

US11	As a support agent, I want to view order and return details.	High	3	Frontend Developer
US12	As an administrator, I want to manage user permissions.	High	5	Backend Developer

Acceptance Criteria:

- Customers can submit valid return requests.
- Support agents can view order and return status.
- Users can access only authorized features.
- Administrative actions are recorded.

Story points are relative estimates, not fixed hours. The estimates are illustrative planning values.

Prioritization Method

The MoSCoW method is used.

Priority	**Meaning**
Must Have	Essential for the complete order fulfilment journey
Should Have	Important but not critical
Could Have	Useful enhancement
Won't Have	Deferred from the initial release

Order placement, stock reservation, payment safety, picking, dispatch, tracking, returns, and refund authorization are Must Have features.

Advanced analytics and AI forecasting are deferred.

Definition of Ready (DoR)

A user story is Ready when:

- The user requirement is clear.
- Acceptance criteria are defined.
- Dependencies are identified.
- Priority is assigned.
- Estimate is available.
- Required test data is prepared.

Definition of Done (DoD)

A user story is Done when:

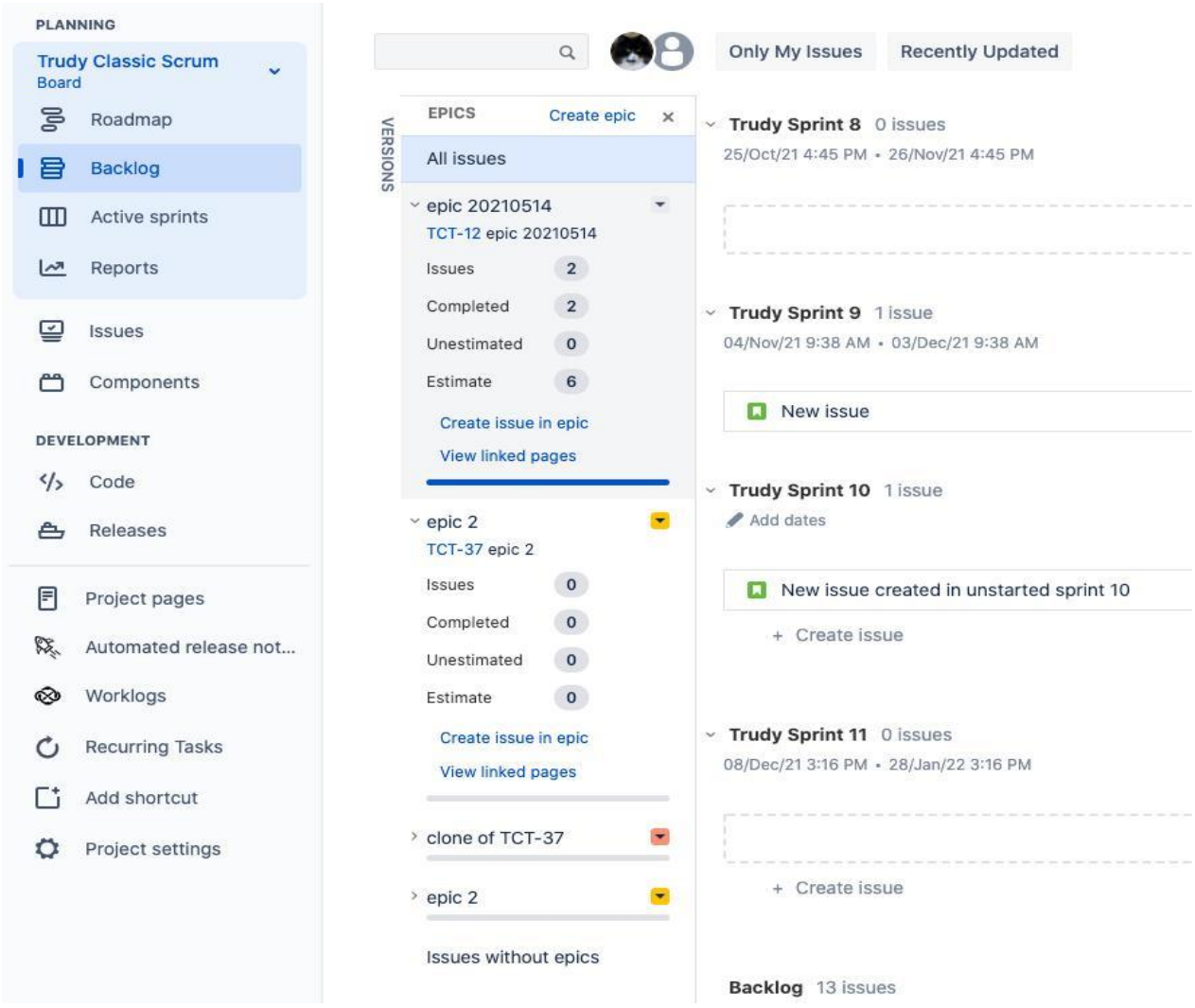
- Development is completed.
- Code review is completed.

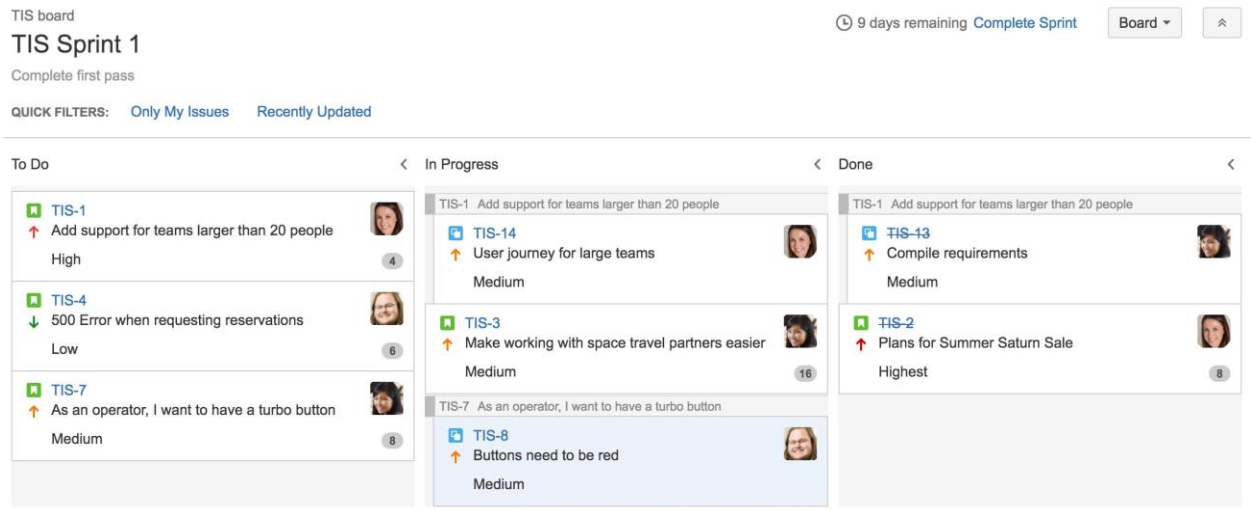
- Unit and integration tests pass.
- Acceptance criteria are satisfied.
- Security and authorization checks pass.
- Database changes are verified.
- Documentation is updated.
- The Product Owner accepts the story.

Stakeholder Priority Decisions

Decision 1: Warehouse pickers require accurate stock reservation, while customers request additional storefront features. Stock reservation is prioritized because it directly prevents failed orders.

Decision 2: Finance reviewers require auditable refunds, while support agents request faster return handling. Refund authorization and audit records are prioritized because they protect financial accountability.


PLANNING
Trudy Classic Scrum
Board
Roadmap
Backlog
Active sprints
Reports
Issues
Components
DEVELOPMENT
Code
Releases
Project pages
Automated release not...
Worklogs
Recurring Tasks
Add shortcut
Project settings
Only My Issues
Recently Updated
VERSIONS
EPICS
Create epic
All issues
epic 20210514
TCT-12 epic 20210514
Issues 2
Completed 2
Unestimated 0
Estimate 6
Create issue in epic
View linked pages
epic 2
TCT-37 epic 2
Issues 0
Completed 0
Unestimated 0
Estimate 0
Create issue in epic
View linked pages
clone of TCT-37
epic 2
Issues without epics
Trudy Sprint 8 0 issues
25/Oct/21 4:45 PM • 26/Nov/21 4:45 PM
Trudy Sprint 9 1 issue
04/Nov/21 9:38 AM • 03/Dec/21 9:38 AM
New issue
Trudy Sprint 10 1 issue
Add dates
New issue created in unstarted sprint 10
+ Create issue
Trudy Sprint 11 0 issues
08/Dec/21 3:16 PM • 28/Jan/22 3:16 PM
+ Create issue
Backlog 13 issues


TIS board
TIS Sprint 1
Complete first pass
9 days remaining Complete Sprint
Board
QUICK FILTERS: Only My Issues Recently Updated
To Do
TIS-1 Add support for teams larger than 20 people
High 4
TIS-4 500 Error when requesting reservations
Low 6
TIS-7 As an operator, I want to have a turbo button
Medium 8
In Progress
TIS-1 Add support for teams larger than 20 people
TIS-14 User journey for large teams
Medium
TIS-3 Make working with space travel partners easier
Medium 16
TIS-7 As an operator, I want to have a turbo button
TIS-8 Buttons need to be red
Medium
Done
TIS-1 Add support for teams larger than 20 people
TIS-13 Compile requirements
Medium
TIS-2 Plans for Summer Saturn Sale
Highest 8

TASK 4 – Create a Capacity-Based Release Plan and Delivery Workflow

Capacity Calculation

Planning assumptions:

- Team size = 7 members.
- Sprint duration = 10 working days.
- Focused working time = 6 hours per person per day.
- Reserve for meetings, interruptions, and uncertainty = 20%.

Nominal capacity:

$$7 \times 10 \times 6 = 420 \text{ hours}$$

Available focused capacity:

$$420 \times 0.80 = 336 \text{ hours}$$

Therefore, the estimated focused capacity is 336 hours per sprint.

Story points are used for relative estimation and are not converted into fixed hours.

Four-Sprint Release Plan

Sprint	Sprint Goal	Main Features
Sprint 1	Establish order and stock foundation	Login, catalogue, cart, order creation, stock reservation
Sprint 2	Complete payment and warehouse workflow	Payment simulation, safe retry, pick lists, dispatch
Sprint 3	Enable tracking and refund workflow	Order tracking, return requests, refund approval
Sprint 4	Validate and prepare release	Gateway integration, scanner integration, security, testing

Dependency Management

The main dependencies are:

- Order creation depends on customer authentication.
- Stock reservation depends on inventory availability.
- Payment confirmation depends on order creation.
- Picking depends on stock reservation.
- Dispatch depends on successful picking.
- Refund processing depends on payment records and authorization.

The payment gateway sandbox becomes available in week four. Therefore, payment simulation is used before week four.

The barcode scanner arrives in week five. Until then, barcode scanning is simulated using manual input or test data.

Work-in-Progress Limits

Workflow Stage	WIP Limit
In Progress	4
Code Review	3
Testing	3
Blocked	2

These limits reduce multitasking and encourage the team to complete existing work before starting new stories.

Handling Urgent and Unfinished Work

- Urgent work must be approved by the Product Owner.
- A critical production issue may interrupt the sprint.
- The team must record the impact of urgent work.
- Unfinished stories return to the product backlog.
- Unfinished stories are re-estimated if requirements change.
- The team does not automatically carry all unfinished stories into the next sprint.

The selected features collectively support the complete order-to-delivery-to-refund jourey.



				Story Points		
Date	Issue	Event Type	Event Detail	Inc.	Dec.	Remaining
02/Nov/2015 12:14 AM	JRA-36493	Sprint start		4		
	JRA-37038			4		
	JRA-37489			2		
	JRA-40856			4		
	JRA-41228			4		
	JRA-43275			4		

TASK 5 – Simulate Delivery and Validate the Design

Predicted Bottleneck

The main predicted bottleneck is payment integration because the payment gateway sandbox is unavailable until week four.

This may delay payment testing and prevent the team from validating the complete order workflow.

Control Rules

- Use a payment simulator during Sprint 1.
- Define a payment API contract before gateway access.
- Use idempotency keys for safe retries.
- Limit WIP to prevent excessive unfinished work.
- Record blocked time in Jira.

Sprint 1 Simulation

Illustrative simulation only. These are not actual Jira results or measured team performance.

Event	Simulated Event	Jira Status
1	Sprint planning completed	To Do
2	Login story started	In Progress
3	Database schema created	In Progress
4	Product catalogue developed	In Progress
5	Order creation dependency identified	Blocked
6	Stock reservation API developed	In Progress
7	Stock validation defect detected	In Progress
8	Defect corrected and tested	Testing
9	Order workflow reviewed	Code Review
10	Login and catalogue accepted	Done
11	Stock reservation validated	Done

5.3 Simulated Delivery Indicators

Assume 16 story points were planned and 11 story points were completed.

Completion Rate

$$\text{Completion Rate}= \frac{11}{16}\times100$$

$$=68.75\%$$

Additional Indicators

Indicator	Simulated Result
Planned story points	16
Completed story points	11

Completion rate	68.75%
Blocked time	12 hours
Average cycle time	3 days
Reopened stories	1
WIP at sprint end	2

Interpretation

The simulation indicates that the team completed 68.75% of its planned work.

The main causes of delay were:

- Database dependencies.
- Stock reservation defects.
- Payment integration uncertainty.
- Limited stakeholder clarification time.

The results suggest that the original plan was slightly optimistic.

Improvement for Sprint 2

The following improvements are introduced:

1. Reduce planned work by 15%.
2. Complete API contracts before development.
3. Increase automated integration testing.
4. Review blocked stories daily.
5. Prioritize payment simulation and safe retry.
6. Conduct earlier stakeholder clarification.

The revised plan reduces uncertainty and improves the probability of completing the sprint goal.

TASK 6 – Redesign Delivery Under Changed Conditions

Changed Requirements

At the end of Sprint 1, the organization introduces the following changes:

- Two additional regional warehouses must use the platform.
- A second seven-member development team joins.
- The original payment engineer becomes unavailable.
- Region-specific tax, shipping, and refund approval rules are required.
- The festival-sale deadline remains unchanged.

Impact Analysis

Area	**Impact**
Scope	Additional regional functionality
Capacity	New team increases capacity but requires onboarding
Payment	Payment expertise becomes unavailable
Dependencies	More coordination between teams

Testing	Region-specific validation required
Deadline	No extension available
Risk	Incomplete integration or delayed release

The second team cannot immediately double delivery capacity because onboarding, coordination, and shared dependencies consume time.

6.3 Redesign Objectives

The revised strategy aims to:

- Preserve the complete order-to-delivery-to-refund journey.
- Support the original and two additional warehouses.
- Maintain the fixed festival-sale deadline.
- Ensure region-specific tax and shipping rules are validated.
- Maintain payment safety and refund auditability.
- Reduce cross-team dependency delays.

Revised Agile Strategy

Selected approach: Scrum with Lean flow controls and selected SAFe-inspired coordination practices.

The project continues using four two-week sprints.

The two teams share a common product backlog, release goal, and integration standards.

Team Responsibilities

Team	**Responsibilities**
Team A – Core Platform	Order management, inventory, payment contracts, refund governance
Team B – Regional Fulfilment	Regional warehouse configuration, tax, shipping, picking, tracking
Shared QA	Integration testing, regression testing, acceptance testing
Product Owner	Priorities and release decisions
Scrum Masters	Team coordination and blocker removal

The payment engineer's absence is managed through shared technical knowledge, documentation, and support from the remaining backend engineers.

Payment work is simplified to the minimum safe functionality required for the release.

Revised Backlog Priorities

Feature	**Priority**	**Decision**
Order creation	Highest	Retain
Stock reservation	Highest	Retain
Payment idempotency	Highest	Retain
Warehouse picking	Highest	Retain
Dispatch and tracking	High	Retain
Refund approval	Highest	Retain

Regional tax rules	Highest	Add
Regional shipping rules	High	Add
Regional refund policies	High	Add
Advanced analytics	Low	Defer
AI demand forecasting	Low	Defer
Loyalty programs	Low	Defer

The redesign protects the complete business journey while removing lower-value features.

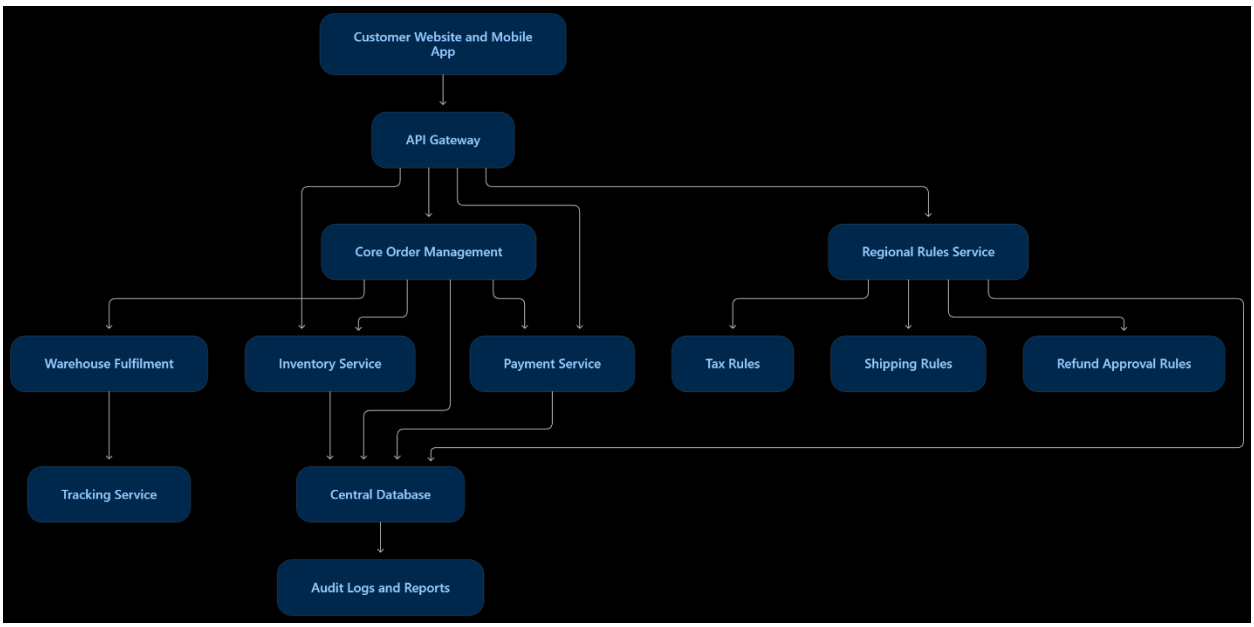
Scaled Agile Coordination

The following coordination practices are adopted:

- Shared product backlog.
- Common Definition of Ready and Definition of Done.
- Weekly cross-team dependency review.
- Shared architecture and API standards.
- Joint sprint reviews.
- Integrated testing before release.
- Common release calendar.

A short cross-team coordination meeting is conducted twice per week.

Revised Architecture



The architecture separates regional rules from the core order workflow. This allows each region to configure its own tax, shipping, and refund policies without duplicating the entire platform.

Revised Simulation

Illustrative simulation of five additional events following the Sprint 1 simulation.

Event	Simulated Event	Result
1	Second team onboarded	Responsibilities assigned
2	Shared API contract completed	Integration dependency reduced
3	Regional tax rules implemented	Regional validation started
4	Payment engineer unavailable	Payment scope simplified
5	Joint integration testing completed	Critical workflow validated

Comparative Validation

The original and revised results below are illustrative planning figures.

Indicator	Original Strategy	Revised Strategy
Planned points	16	20
Completed points	11	16
Completion rate	68.75%	80%
Blocked time	12 hours	7 hours
Reopened stories	1	1
Regional warehouses supported	1	3

Revised Completion Rate

$$\frac{16}{20}\times100=80\%$$

The revised strategy demonstrates an improvement in simulated completion rate from 68.75% to 80%.

The reduction in blocked time indicates that shared API contracts and cross-team coordination can improve workflow.

These figures are illustrative and do not prove actual productivity gains.

Benefits and Limitations

Benefits

- Better coordination between teams.
- Support for additional warehouses.
- Improved regional flexibility.
- Reduced dependency delays.
- Better visibility of work.
- Protection of the complete order fulfilment journey.

Limitations

- Additional coordination overhead.
- Payment expertise remains a risk.
- Regional policies increase testing complexity.
- The unchanged deadline limits scope.
- Shared services may become bottlenecks.

Final Defence

The revised strategy is feasible because it increases delivery capacity while controlling dependencies and reducing lower-priority scope.

The complete order-to-delivery-to-refund journey is retained. Advanced analytics and AI forecasting are deferred to protect the festival-sale release.

TASK 7 – Individual Technical Reflection

Most Significant Project Management Decision

The most significant decision was selecting Scrum with Lean flow management.

Scrum provides structured planning and review activities, while Lean controls work in progress and reduces waiting time.

This combination is suitable for the eight-week release because the project has a fixed deadline and multiple stakeholder priorities.

Evidence-Based Change

The Sprint 1 simulation showed that only 68.75% of planned story points were completed.

The main problems were payment dependencies, stock reservation defects, and delayed clarification.

Based on this evidence, the next sprint was planned with reduced scope, earlier API contracts, and stronger integration testing.

Remaining Vulnerabilities

The final design remains vulnerable to:

- Payment gateway unavailability.
- Payment engineer absence.
- Regional tax and shipping rule errors.
- Database failures.
- Incorrect stock synchronization.
- Increased integration complexity.

These risks require continuous monitoring and testing.

Ethical Considerations

Customer privacy is protected by using synthetic customer and payment data during testing.

Role-based access control ensures that finance reviewers, support agents, and warehouse pickers access only the information required for their responsibilities.

Refund approval remains a human responsibility.

Warehouse workloads should also be monitored to prevent unfair allocation of work.

The platform should provide accessible order status information to customers.

Future Investigation

Before using the strategy with a real development team, I would investigate:

- Actual team velocity.
- Real warehouse stock accuracy.

- Payment gateway reliability.
- Regional tax and shipping requirements.
- Customer privacy requirements.
- Production performance under peak festival-sale traffic.

I would also conduct further testing to validate the complete order-to-refund journey under realistic operational conditions.

CONCLUSION

The Agile delivery strategy provides a structured approach for developing the Omnichannel Order Fulfilment Platform within the fixed eight-week release period.

Scrum with Lean flow management supports iterative development, while Jira provides backlog management, workflow tracking, and reporting.

The prioritized backlog focuses on stock reservation, payment safety, warehouse picking, dispatch, tracking, returns, and refund governance.

The Sprint 1 simulation identifies payment integration and stock reservation as major bottlenecks. The revised strategy addresses these issues through improved coordination, shared API contracts, regional configuration, and reduced lower-priority scope.

The final design supports a complete order-to-delivery-to-refund journey while adapting to the additional warehouses, second development team, and payment engineer unavailability.

Thus, the proposed Agile strategy provides a feasible and adaptable approach for delivering the first release before the festival-sale deadline.